

- Do some work on its own and call the inherited or overridden method of the same name in the base class. We see this in the `__init__()` method of the `Manager` class.

The actual content of the overriding method depends on the situation. If the client makes a call to a method that does not exist in a subclass, the method call will be sent on to the base class. For example, notice that there is no method named `getName()` in the `Manager` class, but it does exist in the `Employee` base class. If a client calls `getName()` on an instance of `Manager`, that call is handled by the base class, `Employee`.

The `payPerYear()` method of the `Manager` class contains this code:

```
if giveBonus:
    pay = pay + (.10 * self.salary) # add a bonus of 10%
    ❹ print(self.name, 'gets a bonus for good work')
```

The instance variable `self.name` was defined in the `Employee` class, but the `Manager` class has no previous mention of it. This demonstrates that instance variables defined in a base class are available for use in methods of a subclass. Here we are calculating the pay for a manager, which works correctly because `payPerYear()` has access to instance variables defined inside its own class (`self.salary`) *and* instance variables defined in the base class (printing using `self.name` ❹).

Test Code

Let's test our `Employee` and `Manager` objects and call methods of each.

File:

EmployeeManagerInheritance/EmployeeManagerInheritance.py

```
# Create objects
oEmployee1 = Employee('Joe Schmoe', 'Pizza Maker', 16)
oEmployee2 = Employee('Chris Smith', 'Cashier', 14)
oManager = Manager('Sue Jones', 'Pizza Restaurant Manager',
                    55000, [oEmployee1, oEmployee2])

# Call methods of the Employee objects
print('Employee name:', oEmployee1.getName())
```